Forecasting Market Volatility Using Hybrid Machine Learning Models: LightGBM and Neural Networks

Aditya Santosh[1], Arun Govind[2], Atheesh Krishnan[3] & Rajeev Surve[4]

Department of Quantitative Finance, Rutgers Business School, Newark, New Jersey

Abstract - In modern financial markets, volatility is both a measure of risk and a signal of opportunity. Accurate short-term volatility forecasting, particularly at the high-frequency level, is vital for traders, risk managers, and automated systems. This study presents a comparative analysis of two advanced modeling techniques—Light Gradient Boosting Machine (LightGBM) and a custom deep learning Neural Network—applied to the task of predicting realized volatility using high-frequency limit order book and trade data. The project pipeline incorporates rigorous preprocessing, extensive feature engineering, and tailored modeling strategies for each architecture. LightGBM leverages hand-crafted features to forecast volatility at a granular level, while the Neural Network model captures temporal and cross-stock dynamics through deep sequence modeling and attention mechanisms. Model performance is assessed using Root Mean Squared Percentage Error (RMSPE) as the primary evaluation metric. Empirical results demonstrate that the Neural Network achieves superior performance, reducing prediction error significantly compared to the gradient boosting model. The study highlights the complementary strengths of feature-based and sequence-based approaches and lays the foundation for further experimentation with hybrid models and deep learning innovations in volatility forecasting.

I. Introduction

Volatility is a cornerstone of modern finance, reflecting the magnitude of price fluctuations and encapsulating the uncertainty inherent in financial markets. As a proxy for risk and a determinant of derivative pricing, volatility plays a central role in investment strategies, risk management frameworks, and algorithmic trading systems. Its predictive power, particularly in short-term horizons, allows market participants to navigate periods of turbulence, adjust exposures, and capitalize on microstructural inefficiencies.

The advent of high-frequency trading and the proliferation of granular order book data have reshaped the landscape of financial modeling. While traditional econometric techniques have offered valuable insights into volatility dynamics, they often struggle to capture the complexity and speed of modern market behavior. The increasing availability of structured high-frequency datasets opens the door for more sophisticated modeling approaches that leverage both temporal dependencies and cross-sectional interactions between financial instruments.

This project investigates the task of forecasting realized volatility over short intraday windows using two distinct machine learning paradigms: LightGBM, a tree-based gradient boosting framework optimized for tabular data, and a custom-designed Neural Network tailored for time series inputs. Both models are trained on rich features derived from high-frequency order book and trade data, and their predictive performance is evaluated using robust statistical metrics. The goal is to understand not only which model performs better, but also why, and under what conditions each model excels.
Through a comparative lens, this study explores the interplay between feature engineering and deep learning, offering practical insights into the modeling of financial volatility in an era increasingly dominated by data-driven decision-making.

II. Problem Statement

In today's algorithmically driven markets, volatility is no longer a slow-moving phenomenon—it emerges rapidly, often within minutes or seconds, triggered by order flow imbalances, liquidity gaps, or microstructural noise. The ability to anticipate these bursts of volatility, especially in high-frequency trading environments, is of paramount importance to traders, market makers, and institutional risk managers alike.

Modern exchanges produce an enormous volume of high-frequency limit order book and trade data, capturing every bid, ask, and executed trade at sub-second intervals. While rich in information, this data is also inherently noisy, sparse, and asynchronous, posing significant challenges for traditional time-series models. Volatility, in particular, is not directly observable; it must be inferred from latent market dynamics and complex interactions between trading entities.

This project addresses the challenge of *forecasting short-term realized volatility—specifically, over 10-minute trading windows—using a combination of limit order book snapshots and executed trade data*. The core problem lies in extracting meaningful signals from this high-dimensional, high-frequency data and mapping them accurately to a volatility target that is inherently stochastic and skewed.

Our objective is two-fold:

- To evaluate the performance of **LightGBM**, a gradient boosting model trained on carefully engineered tabular features.
- To compare it against a custom-designed **Neural Network**, capable of modeling sequential and cross-sectional dependencies in the data.

Ultimately, the aim is to determine which modeling framework is better suited to this task—*not only in terms of predictive accuracy, measured via*

[1] as4385@scarletmail.rutgers.edu [2]ag2360@scarletmail.rutgers.edu , [3] ak2454@scarletmail.rutgers.edu, [4] rs2257@scarletmail.rutgers.edu

Root Mean Squared Percentage Error (RMSPE), but also in terms of interpreting the underlying mechanisms through which each model learns from the data. By analyzing how each approach responds to different feature groups, time dependencies, and volatility regimes, we seek to uncover actionable insights into both model behavior and market dynamics.

III. Exploratory Data Analysis & Feature Engineering

Dataset Overview

The dataset used in this project originates from a high-frequency trading simulation environment, structured to emulate real-world exchange behavior. It consists of 112 distinct stocks, Timestamps divided into time_id intervals, each representing a 10-minute trading window in three main files:

1. train.csv – Contains the target variable: *realized volatility* for each (stock_id, time_id) pair.
2. book_train.parquet – Captures the order book state every second, including the Best bid and ask prices and sizes at two levels & the Derived signals such as spreads, imbalance, and weighted prices
3. trade_train.parquet – Logs all executed trades every second, including trade price, size, and order count

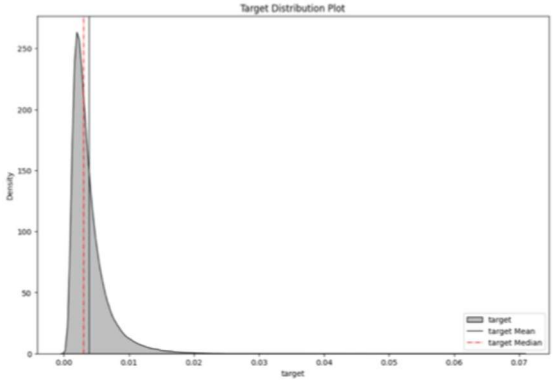
Each entry in the training set corresponds to a specific (stock_id, time_id) pair, with features extracted from order flow and trade activity during that 10-minute window. The prediction target is the realized volatility, defined as the standard deviation of log returns of mid-prices computed per second within the window.

In total, we generated a *training array with 428,932 samples*, each containing a multivariate representation of market behavior at microsecond resolution, enabling rich predictive modeling of short-term volatility patterns.

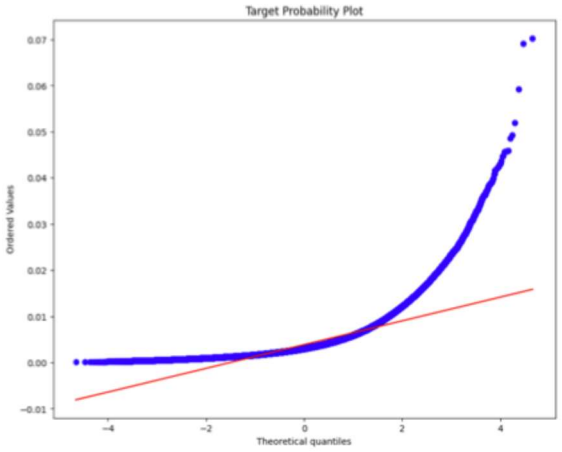
Exploratory Data Analysis (EDA)

Metric	Value
Mean	0.003880
Median	0.003048
Skewness	2.822601
Kurtosis	14.961069

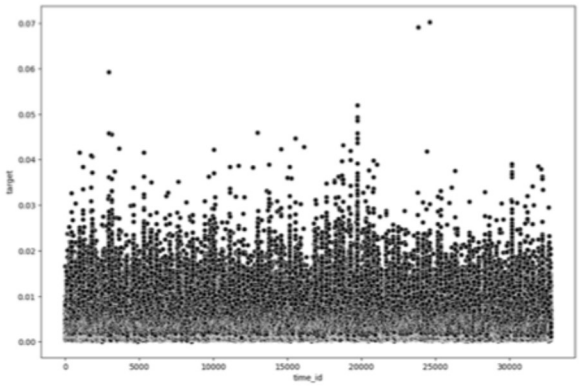
The target variable—realized volatility over a 10-minute window—exhibits a right-skewed distribution, with most values concentrated near zero and a long tail capturing rare but significant volatility spikes. The skewness (2.82) and high kurtosis (14.96) indicate the presence of extreme outliers, consistent with turbulent microstructural events in high-frequency trading.



A Target Plobablity plot (Q-Q plot) reveals substantial deviation from normality, particularly in the upper quantiles, validating the need for models capable of handling non-Gaussian behavior and tail risk. Furthermore, volatility patterns vary significantly across different stocks, with some exhibiting consistent low volatility and others prone to frequent and large price swings. This heterogeneity underscores the necessity of stock-specific feature encoding.



Temporal analysis across time_id shows that volatility events are sporadic and clustered, with no uniform upward or downward trend across the trading session. These clusters often correspond to brief windows of abnormal trading behavior or exogenous shocks—further reinforcing the need for models that adapt to non-stationarity and localized dynamics.



The chart showcases how the order book evolves over time (seconds_in_bucket) and reveals the relationship between quoted prices

[1] as4385@scarletmail.rutgers.edu [2]ag2360@scarletmail.rutgers.edu , [3] ak2454@scarletmail.rutgers.edu, [4] rs2257@scarletmail.rutgers.edu

and executed trades while illustrating the liquidity-taking behavior of market participants.

Throughout the period, the best bid (bid_price1) and best ask (ask_price1) form a fluctuating envelope, representing the narrowest spread at which trades can occur. The second-level bid and ask prices (bid_price2, ask_price2) provide additional context into the depth and resilience of the order book. The deal price, plotted in bold black, weaves dynamically between these quotes, often aligning more closely with either the bid or the ask side, depending on whether buyers or sellers are dominating at that moment.

During certain intervals, particularly around the 200–300 second mark, a clear upward trend is observed across all price lines, reflecting a buy-side imbalance and rising market interest. Later segments exhibit increased volatility, with rapid price reversals and wider spreads, indicating liquidity fragmentation or reaction to microstructural noise.

Feature Engineering : To make sense of the high-frequency order book and trade data, we engineered a comprehensive set of features across both dimensions:

Order Book Features: We extracted signals from the top two levels of the bid and ask prices and sizes, computing features that capture both price informativeness and liquidity asymmetry:

- Weighted Average Prices (WAP1, WAP2)
- Log Returns (log_ret1, log_ret2)
- Spread Metrics (price_spread, bid_spread, ask_spread, wap_balance)
- Liquidity Pressure Features (total_volume, volume_imbalance)
- Aggregation Windows: Features were aggregated over the last 600s, 300s, and 120s to reflect both stable trends and short-term shifts

Initial dataset:

(1507532, 10)

	time_id	seconds_in_bucket	bid_price1	ask_price1	bid_price2	ask_price2	bid_size1	ask_size1	bid_size2	ask_size2
0	5	0	1.000754	1.001542	1.000689	1.001607	1	25	25	100
1	5	1	1.000754	1.001673	1.000689	1.001739	26	60	25	100
2	5	2	1.000754	1.001411	1.000623	1.001476	1	25	25	125
3	5	3	1.000754	1.001542	1.000689	1.001607	125	25	126	36
4	5	4	1.000754	1.001476	1.000623	1.001542	100	100	25	25

Feature dataset:

	log_ret1_std	log_ret2_std	wap_balance_mean	wap_balance_std	price_spread_mean	price_spread_std	bid_spread_mean	bid_spread_std	ask_spread_mean
0	0.000259	0.000403	0.000065	0.000484	0.000852	0.000211	0.000176	0.000162	-0.000151
1	0.000085	0.000175	0.000033	0.000261	0.000394	0.000157	0.000142	0.000148	-0.000135
2	0.000173	0.000351	-0.000138	0.000389	0.000725	0.000164	0.000197	0.000170	-0.000198
3	0.000235	0.000333	0.000199	0.000409	0.000860	0.000280	0.000190	0.000199	-0.000108
4	0.000143	0.000246	-0.000008	0.000317	0.000397	0.000130	0.000191	0.000083	-0.000109

5 rows × 55 columns

Trade Features: Executed trades offer real market flow insights. Key engineered variables include:

- Log Returns of Trade Price (log_ret)
- Trade Size Metrics (size_sum, size_mean, size_std)
- Order Flow Metrics (order_count_sum, order_count_mean, order_count_std)
- Time-Based Aggregation: Features were aggregated over the same multi-resolution windows to maintain temporal consistency

Initial dataset:

(296210, 5)

	time_id	seconds_in_bucket	price	size	order_count
0	5	28	1.002080	553	11
1	5	39	1.002460	8	3
2	5	42	1.002308	147	4
3	5	44	1.002788	1	1
4	5	51	1.002657	100	2

Feature dataset:

	log_ret_std	size_sum	size_mean	size_std	order_count_sum	order_count_mean	order_count_std	log_ret_std_300	size_sum_300	size_mean_300
0	0.000319	3179	79.475000	118.375107	110	2.750000	2.467741	0.000290	1587.0	75.571429
1	0.000165	1289	42.966667	77.815203	57	1.900000	1.446756	0.000151	900.0	56.250000
2	0.000387	2161	86.440000	113.587000	68	2.720000	2.300725	0.000275	1189.0	99.083333
3	0.000387	1962	130.800000	144.828569	59	3.933333	4.043808	0.000381	1556.0	172.888889
4	0.000190	1791	81.409091	117.914682	89	4.045455	4.099678	0.000143	1219.0	110.818182

5 rows × 22 columns

Target Encoding replaced stock_id to capture persistent differences in volatility behavior across stocks with its mean historical target value. This provided the models with a compact representation of stock-specific risk, improving convergence and reducing overfitting. The result was a highly expressive, 76-dimensional feature set per sample, capturing microstructure, execution pressure, and liquidity nuances across time.

IV. Methodology

LightGBM Model

To model realized short-term volatility from high-frequency market data, we implemented a **LightGBM Regressor** — a tree-based gradient boosting algorithm known for its efficiency and scalability on tabular datasets. Its ability to handle sparse features, categorical variables, and non-linear relationships made it an ideal choice for our structured and engineered input space.

Feature Representation: The feature matrix supplied to the LightGBM model consisted of 76 variables per training sample. These features were derived through comprehensive engineering from both the order book and trade book data sources. They included:

- Price dynamics: log returns of weighted average prices (WAPs), spread statistics at multiple depths
- Liquidity indicators: bid-ask volume imbalance, total quoted depth
- Order flow measures: trade size statistics, order count aggregates
- Multi-resolution aggregations: statistical summaries over 600s, 300s, and 120s intervals
- Stock identity: stock_id encoded using its historical average volatility

This design allowed the model to learn from both persistent characteristics of individual stocks and rapidly evolving microstructure signals.

Training Setup: Model development followed a traditional supervised learning pipeline. The key steps included:

- Label Assignment: The target variable was the realized volatility for each (stock_id, time_id) pair, computed from second-wise log returns over a 10-minute window.
- Data Splitting: Cross-validation was conducted using a 3-fold strategy to ensure generalization and robustness.
- Hyperparameter Tuning: A single iteration of RandomizedSearchCV was used to sample hyperparameters from predefined distributions due to computational constraints.

Final Hyperparameter Configuration; The selected configuration for the prototype run included:

- n_estimators: 1900
- max_depth: 11
- num_leaves: 40
- colsample_bytree: 0.2
- subsample: 0.01
- reg_alpha: 1.2575
- reg_lambda: 2.505

These values reflect a balance between model depth and regularization strength, tailored to avoid overfitting on a large, high-dimensional dataset.

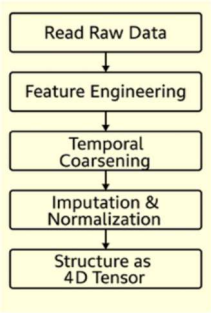
[1] as4385@scarletmail.rutgers.edu [2]ag2360@scarletmail.rutgers.edu , [3] ak2454@scarletmail.rutgers.edu, [4] rs2257@scarletmail.rutgers.edu

Neural Network Model

To overcome the limitations of static, tree-based models in capturing temporal dependencies and cross-stock interactions, we developed a custom Neural Network architecture tailored specifically for high-frequency volatility prediction. The model was designed to operate on multi-stock, time-series inputs, allowing it to learn from both sequential patterns and inter-stock relationships inherent in the data.

Data Representation: Unlike LightGBM, which processes flattened, aggregated features, the neural network ingests structured 4D tensors constructed as: (Batch Size, Stocks, Time Steps, Features) → Shape: (N, 112, 200, 21)
Each tensor captures a temporal snapshot of market microstructure, enabling the network to learn fine-grained patterns in price evolution, liquidity shifts, and order flow pressure over time.

Preprocessing Pipeline:



To prepare the data for model ingestion, we applied temporal coarsening to balance resolution with memory efficiency, then feature engineered, followed by computing dynamic metrics, then applied forward-filling (FFILL) to handle sparse quotes, log1p transformations for skewed features, and Z-score normalization per stock to standardize across scales, finally performed tensor assembly.

Model Architecture:

The architecture is structured to learn from both temporal dynamics and stock-specific behaviors.

Temporal Convolution Layer captures local price and volume patterns over short time horizons.
Stock Embedding Layer encodes stock_id as a learnable vector, enabling the model to capture persistent cross-sectional differences.
GRU Layer processes the temporal signal, preserving long-term dependencies.

Attention Mechanisms: Stock Attention (Enables the model to learn **inter-stock interactions**) & Time Attention (Dynamically weighing earlier vs. later time steps.)

V. Model Evaluation & Results

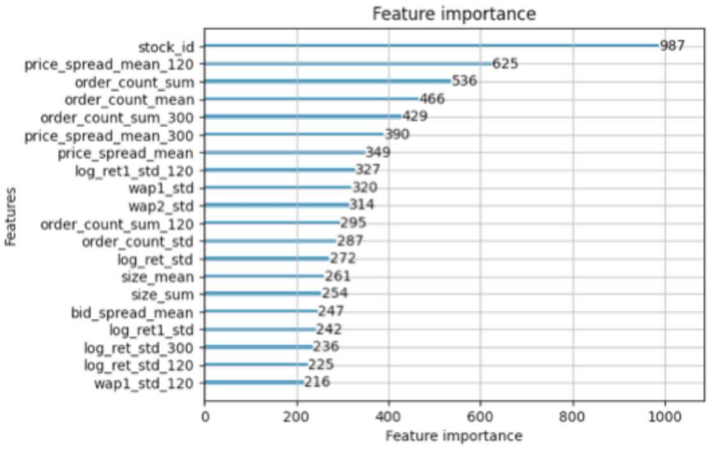
LightGBM Evaluation

To establish a robust baseline for short-term volatility prediction, we employed **LightGBM** (Light Gradient Boosting Machine), a highly efficient gradient boosting framework optimized for large-scale, tabular data. LightGBM operates through a leaf-wise tree growth strategy, which enables it to capture complex feature interactions with minimal computational overhead—an ideal fit for our feature-rich dataset.

Metric	Value
Model	LightGBM Regressor
Cross-Validation Strategy	3-Fold CV
Optimization Strategy	RandomizedSearchCV
Final Best Hyperparameters	
- Best n_estimators	1900
- Best max_depth	11
- Best num_leaves	40
- Best colsample_bytree	0.2
- Best reg_alpha	1.2575
- Best reg_lambda	2.505
- Best subsample	0.01
Training RMSPE	0.3004
Example Test Predictions	[0.002389, 0.002244, 0.002244]

Despite the limited search space, the model identified a highly effective set of parameters. Specifically, the best-performing configuration used 1900 estimators, a maximum tree depth of 11, and 40 leaves per tree. Regularization was handled via reg_alpha = 1.2575 and reg_lambda = 2.505, helping the model mitigate overfitting in the presence of high-dimensional engineered features.

Other notable hyperparameters included a very low subsample ratio (0.01) and colsample_bytree of 0.2, promoting sparse feature selection and reducing noise sensitivity. The model achieved a training RMSPE of 0.3004, indicating a 30% normalized error relative to the target volatility values. When tested on unseen samples, the model produced volatility forecasts such as 0.002389, 0.002244, and 0.002244, demonstrating consistency and stability in output.



The feature importance plot derived from the trained LightGBM model offers crucial insights into which variables most significantly influenced the model's volatility predictions. Unsurprisingly, stock_id emerged as the most influential feature by a wide margin, confirming that different stocksinherently possess distinct volatility profiles. This highlights the need for models to capture cross-sectional heterogeneity across financial instruments.

Among engineered features, short-term spread-based variables like price_spread_mean_120 and price_spread_mean_300 were highly ranked, indicating that tightness or looseness of the order book at recent intervals plays a key role in shaping volatility. These features effectively reflect

[1] as4385@scarletmail.rutgers.edu [2]ag2360@scarletmail.rutgers.edu , [3] ak2454@scarletmail.rutgers.edu, [4] rs2257@scarletmail.rutgers.edu

immediate liquidity and transaction cost pressure, making them valuable indicators of market turbulence.

Neural Network Evaluation

The neural network was implemented in PyTorch, with an input tensor structured as (Batch, Stocks=112, Time Steps=200, Features=21). The architecture begins with two temporal convolution layers—first with a kernel size of 3 to capture local time-window patterns, followed by a kernel size of 1 to fuse features at each time step. Each convolution layer is followed by Layer Normalization to stabilize learning, and the GELU activation function is used throughout for smooth non-linearity.
The model includes two key attention blocks:

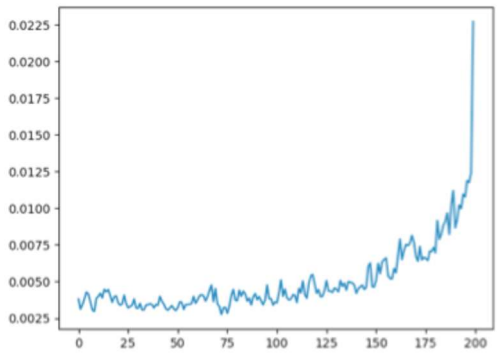
- A Temporal Attention Block to dynamically prioritize critical moments within each 10-minute window
- An Asset Attention Block to model cross-stock dependencies and co-movements

Sequential modeling is handled by a 2-layer GRU (Gated Recurrent Unit) with a hidden size of 32 and a dropout rate of 0.3 between layers. The final output is generated through a single linear head, producing a volatility prediction for each stock within the batch.

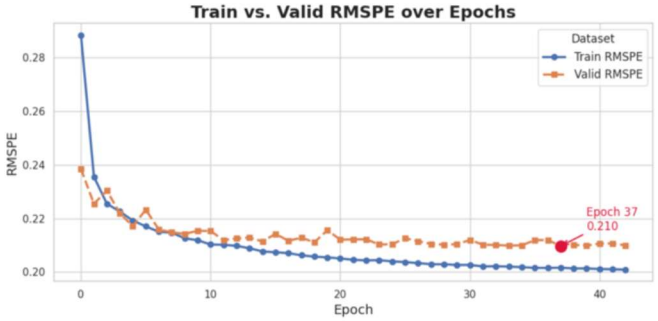
Training was guided by the RMSPE (Root Mean Squared Percentage Error) loss function, which focuses on relative prediction accuracy—a key requirement given the heavily skewed nature of the volatility distribution. The network was optimized using the Adam optimizer (learning rate = 0.001), with an ExponentialLR scheduler that decayed the learning rate by a factor of 0.93 each epoch. Training was capped at 50 epochs, with early stopping enabled to prevent overfitting due to computational constraints.

Component	Type	Parameters
stock_emb	Embedding	3.6K
conv1 + conv2	1D Convolutions	1.8K
norm1 + norm2	LayerNorm	14.4K
rnn	GRU	12.7K
timesteps_attn	Temporal Attention	200
stock_attn	Asset Attention	14.7K
fc_out1	Linear Head	66
Total	—	**47.466K**

The neural network comprises approximately 47.5K trainable parameters, distributed across key components designed for time-series volatility modeling. The GRU module (12.7K) handles temporal dependencies, supported by Temporal Attention (200 params) and Asset Attention (14.7K) to dynamically focus on critical time steps and stock interactions. A final linear head outputs the predicted volatility. This lightweight yet expressive design balances efficiency with deep sequence learning capability.
The neural network's temporal attention mechanism provides interpretability into when during the 10-minute window the model focuses its learning. The plotted attention weights illustrate that the model does not treat each time step equally but instead learns to dynamically assign higher importance to those intervals that are more predictive of future volatility.



From the visualization, it is evident that attention intensifies sharply in the later time steps—especially beyond the 150th step (i.e., towards the final 90 seconds of the window). This behavior validates the effectiveness of the Time Attention module, which enables the model to identify temporal regions of interest rather than relying on naive average pooling.



The model rapidly reduces RMSPE during the initial epochs, dropping from over 0.28 to approximately 0.22 within the first 5 epochs. The best validation RMSPE of 0.210 was achieved at epoch 37, after which early stopping was triggered to prevent overfitting. The narrow and stable gap between training and validation curves indicates good generalization and confirms that the model-maintained robustness across batches without overfitting to noise.

VI. Conclusion

This project compared two approaches for short-term volatility forecasting using high-frequency market data. The LightGBM model, leveraging 76 handcrafted features, achieved a training RMSPE of 0.3004, serving as a strong interpretable baseline. In contrast, the custom neural network—designed with temporal convolutions, GRU layers, and attention mechanisms—reduced the validation RMSPE to 0.210, marking an approximate **30% improvement in relative prediction error**.

The neural network's ability to learn directly from raw temporal sequences enabled it to capture nuanced microstructural patterns and regime shifts that static models could not. Moreover, the incorporation of asset-level and temporal attention allowed the model to dynamically focus on critical time steps and inter-stock relationships, enhancing predictive accuracy in volatile intervals. These results demonstrate the importance of sequence-aware architectures in financial modeling, particularly under high-frequency conditions where traditional methods may fall short.

[1] as4385@scarletmail.rutgers.edu [2]ag2360@scarletmail.rutgers.edu , [3] ak2454@scarletmail.rutgers.edu, [4] rs2257@scarletmail.rutgers.edu